MOBILE APPLICATION DEVELOPMENT

ANDROID FUNDAMENTALS MODULE

Android Web Services & APIs

Understanding APIs, SOAP & REST, JSON, and How Android Apps Talk to Servers



Contents

1. What Is a Web Service?
2. What Is an API?
3. Web Service vs. API
4. Types of Web Services: SOAP vs. REST
5. HTTP Request Methods
6. JSON: The Data Interchange Format
7. JSON vs. XML
8. Parsing JSON & XML in Android
9. Consuming APIs in Android
10. Testing APIs with Postman
11. Creating Mock APIs for Development
12. Putting It All Together
13. Summary

What Is a Web Service?

A web service is a piece of software that lets two applications talk to each other over a network, usually the internet, using standard, agreed-upon rules.



Platform-independent

Works across different operating systems and programming languages.

Network-accessible

The client and server can be anywhere — across the room or across the world.

Standard data formats

Data is usually exchanged as JSON or XML so both sides understand it.

Self-contained requests

Each request carries everything the server needs to respond.

What Is an API?

An API (Application Programming Interface) is a contract that lets one piece of software request something from another — without needing to know how it works inside.

Think of it like ordering food at a restaurant:

1. **You (the Client)**

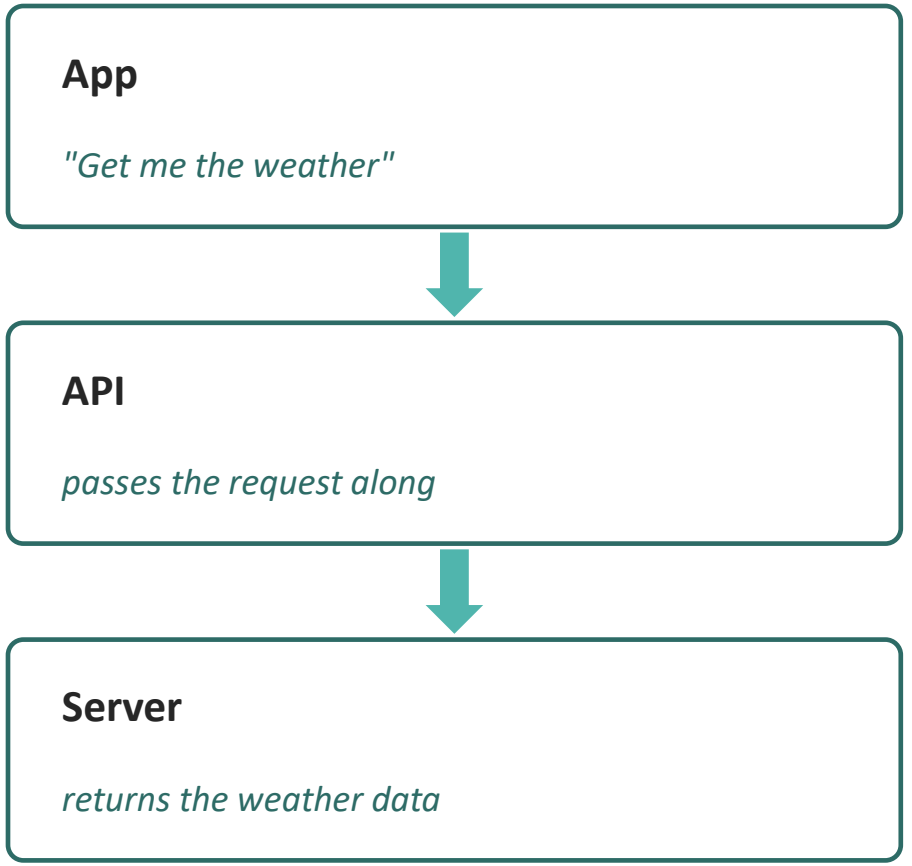
   Place an order — you ask for what you want without knowing how it's cooked.

2. **The Waiter (the API)**

   Carries your request to the kitchen and brings the finished dish back to you.

3. **The Kitchen (the Server)**

   Does the actual work — prepares the data or result you asked for.



Web Service vs. API

Every web service is a kind of API, but not every API is a web service. APIs are the broader idea.

Criteria	API	Web Service
Scope	Any interface that lets software request functionality — over a network or not.	A specific kind of API that always works over a network.
Network required?	Not always — e.g., a local Android SDK call.	Always — a web service has no meaning without a network.
Protocol	No fixed rules — can be any interface design.	Follows a defined protocol, such as SOAP or REST.
Example	Android's Camera API (no internet needed).	A REST endpoint like a weather service at a URL.

Types of Web Services: SOAP vs. REST

1

SOAP

A strict, XML-only messaging protocol with formal rules. Common in banking and enterprise systems.

Statefulness

SOAP can maintain state across calls.
REST is stateless by design.

Message Format

SOAP: XML envelope only.
REST: usually lightweight JSON.

2

REST

A flexible architectural style built on standard HTTP. Supports JSON, XML, or plain text.

Performance

SOAP's XML wrapper adds overhead.
REST/JSON is lighter and faster to parse.

When to Use

SOAP for strict, secure enterprise contracts.
REST for most modern mobile and web APIs.

HTTP Request Methods

REST APIs use standard HTTP methods to express an action — mapping directly to CRUD operations.

GET	**Read**	Retrieve data from the server without changing anything.
POST	**Create**	Send new data to the server to create a resource.
PUT	**Update**	Replace an existing resource entirely with new data.
PATCH	**Update**	Update part of an existing resource.
DELETE	**Delete**	Remove a resource from the server.

JSON: The Data Interchange Format

JSON (JavaScript Object Notation) is the most common format for exchanging data between apps and servers today.

- Lightweight — smaller than XML for the same data
- Human-readable — easy to scan and debug
- Language-independent — supported by virtually every platform
- Maps naturally to objects, arrays, and key-value pairs

```
{
  "city": "Colombo",
  "temperature": 24,
  "conditions": "Cloudy",
  "forecast": [
    { "day": "Mon", "high": 26 },
    { "day": "Tue", "high": 27 }
  ]
}
```

A typical JSON response

JSON vs. XML

Criteria	JSON	XML
Syntax	Key-value pairs with { } and [ ]	Nested tags, like HTML
File Size	Smaller — less markup overhead	Larger — every value needs open/close tags
Readability	Very easy to scan quickly	More verbose, but self-descriptive
Parsing Speed	Faster to parse on mobile devices	Slower — more text to process
Common Use	Most modern REST APIs	SOAP services, legacy systems, config files

Parsing JSON & XML in Android

1

org.json (Built-in)

JSONObject and JSONArray ship with Android — no extra dependency needed.

2

Gson (Google)

Converts JSON directly into Kotlin/Java model classes using annotations.

3

Moshi (Square)

A modern, Kotlin-first JSON library with adapters and null-safety.

4

XmlPullParser

Android's native, memory-efficient parser for XML — recommended for most cases.

5

SAX Parser

Event-driven XML parsing; low memory, good for large files.

6

DOM Parser

Loads the whole XML tree into memory; simplest API, least efficient.

Consuming APIs in Android

Retrofit

Type-safe HTTP client that turns REST APIs into Kotlin/Java interfaces. Pairs with Gson or Moshi for parsing. Best fit for most modern apps.

OkHttp

Low-level HTTP client handling connections, caching, and interceptors. Often runs underneath Retrofit.

Volley

Google's earlier networking library. Simple for small request queues but less flexible for large-scale apps today.

Every request follows the same pattern: build the request → send it over HTTP → receive a status code and payload → parse the payload → update the UI.

Testing APIs with Postman

Postman is a tool that lets developers send API requests and inspect responses — without writing any app code first.

- Verify an endpoint works before writing any Android code
- Inspect the exact JSON or XML structure the server returns
- Save and share request collections with your team
- Automate simple checks and repeat tests quickly

```
GET  /api/weather/colombo
```

```
Status: 200 OK

{
  "city": "Colombo",
  "temperature": 24
}
```

Sending a request and reading the response in Postman

Creating Mock APIs for Development

A mock (fake) API returns realistic sample data without a real backend — useful while the actual server is still being built.

- Build and test the Android UI before the real backend is ready
- Safely simulate error responses, like 404 or 500, without breaking anything real
- Keep working during development even when offline

json-server

Turns a simple JSON file into a full working REST API in seconds.

MockAPI.io

A hosted, no-code tool for building fake endpoints in the browser.

Postman Mock Servers

Generate a mock endpoint straight from a saved Postman collection.

Putting It All Together

A typical workflow, from designing an API to shipping it inside an Android app:

1. **Design the API contract**
   Decide the endpoints, methods, and JSON shape.
2. **Test it in Postman**
   Send sample requests and confirm the response format.
3. **Build against a mock API**
   Develop the Android UI before the real backend exists.
4. **Swap in the real API**
   Point the app at the live endpoint once it's ready.
5. **Parse & display in the app**
   Use Retrofit/Gson (or XmlPullParser) to show the data.

Summary

1. A web service lets apps talk to each other over a network; an API is the broader contract for requesting functionality
2. SOAP is strict and XML-only; REST is flexible and usually uses lightweight JSON
3. HTTP methods (GET, POST, PUT, PATCH, DELETE) map directly to CRUD operations
4. JSON is the most common data format today — smaller and faster to parse than XML
5. Postman and mock APIs let you design, test, and build against an API before the real backend is ready

Thank You

Questions